```
> y p
--- !ruby/object:Project
concise_attributes:
- !ruby/object:ActiveRecord::Attribute::FromDatabase
  name: id
- !ruby/object:ActiveRecord::Attribute::FromUser
  name: name
  value_before_type_cast: Project Runway
  original_attribute: !ruby/object:ActiveRecord::Attribute::FromDatabase
    name: name
    type: !ruby/object:ActiveModel::Type::String
      precision:
      scale:
      limit:
<there's more here>
```

```
> ap p
#<Project:0x00000102ff3568> {
            :id => nil,
          :name => "Project Runway",
      :due_date => Sat, 16 Aug 2014,
    :created_at => nil,
    :updated_at => nil,
        :public => false
}
```

There's a nice progression here, from `puts` just giving the class name, to `p` giving a class name and key/value pairs, to `y` putting each key/value on a different line, to Awesome Print lining up the values for readability. The Awesome Print output is also a syntactically correct Ruby hash, should you feel the need for copy and paste.

Prescription 41 Using `p` and various related methods is a quick and easy way to get a sense of why a test is behaving badly.

Using these print statements in a successful troubleshooting section is a tug-of-war between finding interesting data and not overwhelming yourself.

One catch to keep in mind: all these kernel method calls work fine from a test console or in the server log if you run Rails via `rails server`. If you use a development server, such as Unicorn, that doesn't output STDOUT to the terminal, then you need to use the Rails logger to see printed output. The Rails logger is available anywhere in your application with `Rails.logger`, which then takes one of several methods, such as `Rails.logger.error`, all of which take an argument that gets printed to the log or a block whose final value gets printed to the log.